

Script thuyết trình — Banking-grade AI Engineering (~45 phút)

Slide 01 · Cover

Chào mọi người. Hôm nay mình sẽ demo một project tên là AI Agent Skill Prompt — hay gọi tắt là AI_SKILLS.

Mục tiêu không phải là giới thiệu thêm một tool AI mới. Mà là cho mọi người thấy cách chuẩn hóa lại việc dùng GitHub Copilot trong môi trường ngân hàng — có rule, có quy trình, có audit trail — thay vì để mỗi người dùng theo cách riêng.

Slide 02 · Agenda

Buổi hôm nay chia làm 8 phần:

Phần 1 là bối cảnh — tại sao banking đặc thù và AI tự do có rủi ro gì. Phần 2 là AI Agent — agent là gì, khác chat hoặc autocomplete thế nào. Phần 3 là architecture — cấu trúc package gồm Instructions, Agents, Skills. Phần 4 là demo flow thực tế — đi qua một task từ Plan đến Docs. Phần 5 là giá trị cho developer hằng ngày. Phần 6 là giá trị cho Team. Phần 7 là so sánh Freestyle AI với Governed AI. Phần 8 là KPI, roadmap và Q&A.

Tổng khoảng 45 phút, cuối buổi có thời gian hỏi đáp.

Slide 03 · Mục tiêu

Sau buổi này, mình mong mọi người nắm được 5 điểm:

Một là hiểu vì sao Copilot mặc định chưa đủ cho môi trường banking. Hai là hiểu AI Agent là gì và khác gì so với chat hay autocomplete thông thường. Ba là nắm được cấu trúc package: instructions, prompts, agents, skills, docs. Bốn là xem được demo flow thực tế từ Plan đến Implement, Review, Test rồi Docs. Năm là thấy rõ giá trị cụ thể cho developer và cho lead hoặc manager.

Target của buổi này là engineer và tech lead trong team SmartSales.

SECTION 01 — Bối cảnh Banking

Slide 05 · Đặc thù Banking

Vậy banking có gì khắt khe hơn project thường?

Điểm thứ nhất là PII và sensitive data. Thông tin khách hàng, số tài khoản, số thẻ, secret key — những thứ này không được phép lộ ra trong log, trong response, hay trong bất kỳ output nào của AI.

Điểm thứ hai là business logic phức tạp. một lỗi nhỏ có thể ảnh hưởng trực tiếp đến khách hàng. Không giống như thay đổi màu button mà deploy lại là xong.

Điểm thứ ba là auditability. Mọi thay đổi phải traceable, phải có khả năng rollback. Đây là yêu cầu từ compliance, không phải tùy chọn.

Điểm thứ tư là codebase legacy. Team đã có component chuẩn được approved, có pattern nội bộ, có constraint riêng. Không phải muốn dùng gì cũng được.

Tóm lại: code đúng cú pháp chưa đủ — cần đúng chuẩn ngân hàng.

Slide 06 · Rủi ro AI tự do

Vậy khi developer dùng AI mà không có guardrails/chặn thì chuyện gì xảy ra?

Thứ nhất là AI tạo helper hoặc service mới dù codebase đã có component chuẩn. Dev không biết, AI cũng không biết.

Thứ hai là tự thêm package, tự chọn pattern ngoài chuẩn của team.

Thứ ba là bỏ qua business rule và audit log quan trọng của domain.

Thứ tư là log nhầm PII, token, account data hoặc card data.

Thứ năm là refactor quá rộng — blast radius lớn, không kiểm soát được phạm vi ảnh hưởng.

Thứ sáu là không update tài liệu, không ghi rollback, không chứng minh đã test.

Thứ bảy là claim "done" nhưng thực ra chưa compile, chưa lint, chưa chạy test.

Điều đáng chú ý là AI không nguy hiểm vì code dở. AI nguy hiểm vì code nhanh nhưng không bị ràng buộc bởi rule nội bộ. Không ai dạy nó biết chuẩn banking của team mình.

Slide 07 · Giải pháp: 5 Layer

Để hiểu solution, mình cần nhìn vào cách AI coding tool hoạt động.

Bất kỳ tool nào như Copilot, Cursor hay ChatGPT đều có 4 layer built-in sẵn.

Layer 1 là Model — đây là bộ não, sinh code, reasoning, trả lời câu hỏi. Claude, GPT, Gemini đều nằm ở đây.

Layer 2 là Context Engine — đọc file, index workspace, dùng RAG để hiểu codebase của bạn.

Layer 3 là Tool và Action — sửa file, chạy test, chạy lệnh terminal.

Layer 4 là Apply và Diff — AI đề xuất thay đổi, show diff, bạn xem rồi accept hoặc reject trước khi apply vào workspace. Đây là workspace operation, không phải git.

Cả 4 layer này Copilot, Cursor hay ChatGPT đều đã làm tốt.

Vấn đề là layer thứ 5 — Governance Layer. Đây là nơi chứa rule banking nội bộ của team, chuẩn component, pattern approved, yêu cầu về audit, về PII, về rollback. Không tool thị trường nào tự biết được những thứ này. Chỉ team mình mới định nghĩa được.

AI_SKILLS tập trung vào layer thứ 5. Đó là phần duy nhất không thể mua sẵn.

SECTION 02 — AI Agent

Slide 09 · AI Agent vs Chat

Vậy AI Agent là gì và khác gì so với chat thông thường?

Chat là bạn hỏi, AI trả lời. Mỗi lượt là độc lập. AI không nhớ context lâu dài, không tự làm gì cả.

Agent thì khác. Bạn giao một goal — ví dụ "analyze requirement này và lên plan" — agent tự đọc codebase, trace flow, dùng tool, rồi trả về kết quả có evidence. Nó goal-driven, context-aware và evidence-based.

Sự khác biệt cốt lõi: chat trả lời câu hỏi, còn agent hoàn thành workflow.

Slide 10 · Workflow mới

Với agent, cách làm việc thay đổi hoàn toàn.

Thay vì giao câu lệnh rời rạc như "viết cho tôi function này", bạn giao cả workflow: Analyze → Plan → Implement → Review → Test → Docs.

Có 3 nguyên tắc quan trọng trong workflow này.

Một là Analyze trước, code sau. AI phải đọc codebase, trace flow, hiểu blast radius trước khi đề xuất bất kỳ thay đổi nào.

Hai là Plan phải có evidence. Plan không phải là danh sách công việc chung chung. Plan phải liệt kê affected files, business flow, security risk, rollback và verification commands.

Ba là Done phải có chứng minh. Compile được, lint pass, test pass, docs update — hoặc ghi rõ lý do tại sao chưa làm được.

Trong banking, token rẻ hơn incident. Bỏ thêm context và rule ở đầu vào để đổi lấy output ổn định, ít phải sửa lại.

SECTION 03 — Architecture

Slide 12 · Package map

Project này là một package nằm hoàn toàn trong thư mục `.github/`. Không cần cài thêm gì, không phụ thuộc infrastructure riêng. Copy vào repo nào cũng dùng được.

Package có 5 thành phần.

Thứ nhất là `copilot-instructions.md` — một file duy nhất, là rule nền bắt buộc cho toàn bộ repo.

Thứ hai là thư mục `agents/` với 9 file — mỗi file là một vai chuyên môn như Planner, Reviewer, Tester.

Thứ ba là thư mục `prompts/` với 21 file — mỗi file là một workflow chuẩn hóa như plan, review, fix, docs.

Thứ tư là thư mục `skills/` với 15 file — mỗi file là domain knowledge sâu về banking, security, testing.

Thứ năm là thư mục `docs/` và `copilot/` — knowledge base giúp AI hiểu team đang làm gì, theo chuẩn nào.

Slide 13 · Instructions

`copilot-instructions.md` là file quan trọng nhất. Nó được Copilot đọc tự động cho mọi conversation trong repo.

File này ép Copilot hoạt động như một senior engineer trong regulated environment, với 8 rule chính.

Một là Reference-first: nếu có code cũ, diff hay snippet liên quan thì phải đọc trước. Hai là System-analysis first: phải map architecture và blast radius trước khi code. Ba là Blocked-rules first: đây là rule cấm, có priority cao nhất — không được vi phạm. Bốn là Component-first: tìm component có sẵn trong codebase trước khi tạo mới. Năm là Plan và approval: mọi high-risk change phải có plan trước. Sáu là No silent code: phải nêu rõ affected files, risk và rollback. Bảy là Evidence required: không được claim "done" khi chưa compile và test. Tám là Banking data safety: không expose PII, secrets, card hay account data.

Slide 14 · Agents

9 agent được thiết kế như một engineering team hoàn chỉnh.

Planner phụ trách lên plan, xác định scope, risk, rollback và verification. System Analyst trace flow, phân tích data contract và blast radius. Code Reviewer review với severity-ranked findings về business logic, security và test. Security Reviewer kiểm tra PII, auth, authz và secrets. Tester xây dựng test strategy và tìm missing test cases. Debugger phân tích root cause — không fix mò. Docs Manager giữ docs luôn cập nhật và đồng bộ knowledge. Researcher làm deep research và hỗ trợ ra quyết định. Research Architect so sánh architecture và phân tích tradeoff.

Cách dùng trong VS Code: gõ `@Planner` hoặc `@Tester` trong chat để trigger đúng agent.

Slide 15 · Prompts + Skills + Docs

Ba thành phần còn lại hỗ trợ nhau.

Prompts là 21 workflow chuẩn hóa. Developer không cần tự nghĩ prompt từ đầu mỗi lần. Có sẵn plan, implement, review, line-review, logic-check, fix, test, db-schema, docs-update, git và nhiều hơn nữa.

Skills là 15 domain knowledge files. Khi AI dùng skill Banking Grade Engineering, nó biết các chuẩn banking cụ thể. Khi dùng Secure Code Review, nó biết checklist security cho banking. Developer không cần nhắc lại từ đầu mỗi lần.

Docs là nguồn context. Architecture guideline, banking domain rule, workflow playbook, skills index, agent catalog, changelog — tất cả nằm trong repo. Docs tốt thì AI output tốt hơn, vì AI hiểu team đang làm gì.

SECTION 04 — Demo Flow

Slide 17 · Demo Flow Overview

Bây giờ mình vào phần demo thực tế.

Scenario: một task banking thực tế — ví dụ cập nhật API liên quan đến customer limit hoặc transaction history. AI không sửa code ngay. Phải đi qua 5 bước, mỗi bước có evidence rõ ràng.

Bước 1 là Plan — phân tích impact, liệt kê affected files, business flow, risk, rollback. Bước 2 là Implement — code theo đúng plan đã approve, theo convention, validation, logging. Bước 3 là Review — banking-grade review, severity-ranked, blocking vs suggestion. Bước 4 là Test — happy path cộng edge cases cộng banking-specific risks. Bước 5 là Docs — update business flow, API contract, validation rules.

Slide 18 · Step 1: Plan

Prompt mình dùng: "Analyze this requirement based on our banking rules. Do not implement yet."

Câu cuối quan trọng — "Do not implement yet" — để agent chỉ analyze và plan, không code ngay.

Plan output phải có đủ 7 phần.

Một là affected files — danh sách file cần thay đổi: controllers, services, DB, config. Hai là business flow — luồng nghiệp vụ bị ảnh hưởng, tiền chảy thế nào, trạng thái đổi ra sao. Ba là validation rules — data constraints cần enforce. Bốn là security và privacy risk — PII nào liên quan, auth check nào cần thêm. Năm là test cases outline — happy path, edge cases và banking risk. Sáu là docs impact — tài liệu nào cần update sau khi implement. Bảy là rollback strategy — cách undo nếu có sự cố sau deploy.

Sau khi AI ra plan, developer đọc, kiểm tra, approve rồi mới cho implement. Developer vẫn kiểm soát quyết định cuối.

Slide 19 · Steps 2–5

Bốn bước còn lại:

Implement: AI code theo đúng plan đã approve. Theo architecture convention của team. Có validation, error handling, audit log. Không đổi gì ngoài scope. Giải thích từng file đã sửa.

Review: Kiểm tra architecture correctness, business logic accuracy, security và privacy. Findings được xếp theo severity. Phân biệt rõ blocking issue với suggestion. Đưa ra danh sách required fixes.

Test: Viết test cho happy path, validation cases, permission boundary, edge và error cases, idempotency check và concurrency risk.

Docs: Update business flow, technical behavior, API contract, validation rules, testing notes. Nếu không có gì cần update thì ghi rõ N/A và lý do.

AI không thay reviewer người — nhưng bắt developer tự chuẩn bị PR kỹ hơn trước khi gửi senior review.

SECTION 05 — Giá trị

Slide 21 · Before vs After

Để thấy rõ sự khác biệt, so sánh trực tiếp cùng một task: "Write API update customer limit."

Freestyle AI: tạo controller và service mới, bỏ qua existing pattern. Không check permission, không có audit log. Không xử lý concurrency, không idempotent. Không update docs, không có rollback plan. Output khác nhau mỗi lần, khó audit. Senior reviewer phải bắt lỗi từ đầu.

Governed AI với AI_SKILLS: đọc codebase, tìm existing component trước. Plan có affected files, security risk, rollback. Implement đúng convention, validation, audit log. Review tự động trước khi tạo PR. Test coverage cho banking-specific risks. Docs và evidence đi kèm mỗi thay đổi.

Dùng AI_SKILLS không làm developer bị kiểm soát nhiều hơn. Nó giúp developer ít bị review reject hơn và an toàn hơn trong domain ngân hàng.

Slide 23 · Dev + Leader Value

Với developer hằng ngày: Có sẵn prompt chuẩn, không cần tự nghĩ từ đầu mỗi lần. Checklist rõ ràng cho từng loại task. AI nhắc component-first trước khi tạo mới. Chuẩn bị PR kỹ hơn, ít bị review reject. Member mới onboard nhanh hơn. Senior giảm thời gian review lỗi cơ bản.

Với Tech Lead, CTO và PM: Tận dụng tốt license GitHub Copilot đã trả tiền. Chuẩn hóa cách cả team dùng AI — không mỗi người một kiểu. Giảm rủi ro AI sinh code không phù hợp banking. Tăng chất lượng PR, ít lỗi cơ bản hơn. Giảm knowledge loss khi người cũ rời team. Có nền tảng để scale sang nhiều repo và project.

Đây là AI governance package — không phải bộ prompt cá nhân.

SECTION 07 — So sánh

Slide 25 · AI Tool Landscape 2026

Hiện tại có 4 tool đang nổi:

GitHub Copilot: tích hợp sẵn trong GitHub và VS Code, hỗ trợ instructions, prompts và skills, phù hợp nhất nếu công ty đã có license.

Claude Code: agentic coding qua terminal, IDE và web, đọc được toàn bộ repo, sửa nhiều file cùng lúc, MCP rất mạnh, nhưng cần license riêng.

Cursor: AI-first IDE với codebase indexing, cloud agents, checkpoint — nhưng đổi IDE sẽ tốn adoption cost.

OpenAI Codex: multi-agent song song, sandbox, long-running tasks — nhưng cần Codex plan riêng.

AI_SKILLS không cạnh tranh với các tool trên. Nó là governance layer áp lên bất kỳ tool nào. Tool có thể đổi — governance phải thuộc về team, không phụ thuộc vào vendor.

Hiện tại dùng Copilot là hợp lý vì công ty đã cấp sẵn license.

SECTION 08 — KPI & Q&A

Slide 27 · KPI + Roadmap

Để đo hiệu quả, mình đề xuất 6 KPI:

Một là số PR review findings — so sánh repeated findings trước và sau pilot 2 đến 4 tuần. Hai là lead time — thời gian từ ticket đến PR ready, đo trước và sau. Ba là số PR bị comment thiếu test, docs hoặc rollback. Bốn là onboarding speed — dev mới hiểu flow nhanh hơn qua explain và analyze prompts. Năm là risk caught early — số issue phát hiện sớm bởi security và line review prompt. Sáu là AI adoption quality — số developer dùng prompt và agent chuẩn thay vì prompt tự do.

Roadmap chia 3 phase:

- Phase 1 Adoption: viết README và quickstart cho developer, xác định 5 prompt nên dùng mỗi ngày, demo script 15 phút, PR checklist gồm plan, test và docs.
 - Phase 2 Mapping: cập nhật rule theo architecture thực tế của team, lập approved components list, xây blocked patterns catalog, viết domain-specific skills.
 - Phase 3 Scale: PR comment template AI, risk assessment template, pilot metric dashboard, triển khai multi-repo governance.
-

Slide 28 · Closing

Để tóm lại buổi hôm nay, có 5 thông điệp cốt lõi:

- Một: AI tool hiện tại đều đủ mạnh. Khác biệt là ai có guardrails tốt hơn.
 - Hai: AI_SKILLS là governance layer gồm 5 thành phần: Instructions, Prompts, Agents, Skills và Docs.
 - Ba: Nhanh hơn nhưng vẫn có rule, review, test, docs và audit trail đầy đủ.
 - Bốn: Tận dụng Copilot license sẵn có, không cần đổi tool hay đầu tư thêm infrastructure.
 - Năm: Rule banking phải thuộc về team, không phụ thuộc vào tool vendor. Khi công ty chuyển sang tool khác, governance vẫn còn đó.
- Đây không phải project prompt cá nhân. Đây là AI engineering process cho môi trường banking — có thể review, improve và scale cùng với team.
- Mình mở Q&A. Ai có câu hỏi không?
-

Navigation nhanh

Slide	Section
01	Cover
04	Bối cảnh Banking

Slide	Section
08	AI Agent
11	Architecture
16	Demo Flow
20	Giá trị
22	Leader Value
24	So sánh
26	KPI & Q&A
28	Close